

Relatório do projeto

Sistema de Delivery — Projeto de Estrutura de Dados

Aluno	Daniel Queiroz Soares
Disciplina	Estrutura de Dados
Tema	Sistema de Delivery

1. Descrição do Sistema

O sistema desenvolvido é um **gerenciador de delivery** (entrega de comida), que resolve um problema real enfrentado por restaurantes e lanchonetes: **organizar o cadastro de clientes, o cardápio de produtos, a ordem de preparo dos pedidos e o histórico de entregas** de forma justa e rastreável.

Na prática, quando um restaurante recebe vários pedidos ao mesmo tempo, precisa garantir que sejam preparados na ordem de chegada, acompanhar a evolução do status de cada pedido (e poder desfazer um status lançado por engano) e consultar pedidos antigos navegando para frente e para trás. Cada uma dessas necessidades foi mapeada para a estrutura de dados mais adequada, estudada nas aulas 1 a 9.

O projeto é totalmente **modularizado**: a pasta `modelos/` guarda as classes de dados (Cliente, Produto, Pedido); a pasta `estruturas/` contém as cinco estruturas de dados; a pasta `sistema/` reúne as regras de negócio; e o `main.py` apresenta o menu interativo.

2. Estruturas Utilizadas

Lista Sequencial — Cadastro de Clientes

Os clientes são armazenados em uma **lista sequencial** (`ListaSequencialClientes`), em que os elementos ficam em posições contíguas. **Por quê**: o cadastro de clientes é majoritariamente acessado por consulta direta e percorrido por completo nas listagens; a lista sequencial é simples, ocupa pouca memória extra e oferece inserção rápida no final. A remoção foi implementada manualmente com o deslocamento dos elementos, demonstrando o funcionamento interno da estrutura.

Lista Encadeada — Cardápio de Produtos

O cardápio usa uma **lista encadeada simples** (`Cardapio` + classe `No`), na qual cada produto fica em um nó que aponta para o próximo. **Por quê**: o cardápio muda com frequência (itens entram e saem); na lista encadeada a inserção e a remoção não exigem deslocar todos os elementos como na lista sequencial — basta religar os ponteiros, tornando essas operações mais flexíveis.

Lista Duplamente Encadeada — Histórico de Pedidos

Os pedidos já entregues vão para o **histórico** (`HistoricoPedidos`), uma lista duplamente encadeada em que cada nó aponta para o próximo e para o anterior. **Por quê**: permite **navegar para frente e para trás** entre os pedidos (como botões “próximo/anterior”), algo impossível na lista simples, ideal para o operador revisar o histórico em qualquer direção.

Pilha — Status do Pedido (desfazer)

Cada pedido guarda seu histórico de status em uma **pilha** (Pilha, padrão LIFO). O topo é sempre o status atual. **Por quê:** a regra de “**desfazer o último status**” segue a lógica do último que entra ser o primeiro a sair: se um status foi lançado por engano, basta desempilhar para voltar ao anterior. As operações empilhar, desempilhar e topo modelam isso de forma natural.

Fila — Pedidos em Preparo

Os pedidos recebidos entram em uma **fila** (Fila, padrão FIFO). **Por quê:** em um delivery, o **primeiro pedido a chegar deve ser o primeiro a ser preparado**. A fila garante essa ordem justa de atendimento com as operações enfileirar, desenfileirar e consulta do primeiro elemento.

3. Complexidade

A notação Big-O abaixo descreve como o tempo de execução cresce conforme aumenta a quantidade **n** de elementos.

Operação	Lista Sequencial	Lista Encadeada	Lista Dupla	Pilha	Fila
Inserção	$O(1)$ no final	$O(n)$ no final*	$O(1)$ no final	$O(1)$	$O(1)$
Busca	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Remoção	$O(n)$	$O(n)$	$O(1)$ nas pontas	$O(1)$ topo	$O(n)$ início**

* Na implementação, a inserção no final percorre a lista até o último nó. ** A remoção no início da fila desloca os demais elementos.

Inserção

- **Lista sequencial:** inserir no final é $O(1)$, pois apenas ocupa a próxima posição livre.
- **Lista encadeada / dupla:** criar o nó é $O(1)$; como inserimos no final, percorrer até o último nó custa $O(n)$ (na lista dupla usamos a referência *tail*, tornando-a $O(1)$).
- **Pilha e fila:** empilhar e enfileirar são $O(1)$, pois sempre atuam em uma das pontas.

Busca

- Em todas as estruturas a busca é **sequencial**: percorre elemento por elemento comparando a chave (id/código), portanto $O(n)$ no pior caso.
- Pilha e fila normalmente não são usadas para busca, mas consultar o topo/primeiro é $O(1)$.

Remoção

- **Lista sequencial:** $O(n)$ — após achar o elemento, é preciso deslocar os seguintes uma posição para trás.
- **Lista encadeada:** $O(n)$ para localizar; religar o ponteiro é $O(1)$.
- **Lista dupla:** com o nó em mãos, a remoção é $O(1)$ (ajusta anterior/próximo); remover nas pontas é direto.
- **Pilha:** desempilhar (remover o topo) é $O(1)$.
- **Fila:** desenfileirar remove o primeiro; nesta implementação custa $O(n)$ pelo deslocamento, mantendo a ordem FIFO.